■ AGRIMIND

Multi-Agent AI Soil, Crop & Fertilizer Optimization System

Project Description & Technical Architecture

A complete overview of what we are building, the problem being solved, the AI agents, RAG pipeline, APIs, frameworks, architecture, features, and development roadmap.

1. Project Overview

AgriMind is an AI-powered agricultural decision-support system that analyzes a farmer's soil report, combines authoritative agricultural knowledge with live weather data, and uses specialized AI agents to recommend suitable crops, fertilizers, natural soil amendments, application timing, and explain why each recommendation was made.

This is not intended to be a simple chatbot. The system combines RAG, AI agents, soil-data analysis, a deterministic recommendation/optimization layer, live weather APIs, an LLM, and a web application.

2. Problem We Are Solving

A farmer may receive a Soil Health Card or soil laboratory report containing values such as nitrogen, phosphorus, potassium, pH, organic carbon, electrical conductivity and micronutrients. The farmer still needs practical answers: Which crop is suitable? What nutrient is deficient? Which fertilizer or soil amendment should be considered? Can an organic alternative be used? When is an appropriate application window? How does upcoming weather affect the decision? And why was a particular recommendation made?

AgriMind aims to bridge the gap between raw soil-test information and an understandable, evidence-grounded farming decision.

3. What the System Takes as Input

• **Soil report:** PDF containing soil-test values.

• **Location:** city/region or ideally latitude and longitude.

• **Crop:** a selected crop, or a request asking which crop is suitable.

• **User question:** for example, 'What fertilizer should I apply this week?'

The system combines soil, crop, location, season, live weather, and agricultural knowledge before generating a recommendation.

4. Two-Agent Architecture

Agent 1 — Soil & Crop Recommendation Agent

Agent 1 handles the soil and crop side of the problem. Its responsibilities include soil analysis, nutrient-deficiency detection, crop suitability, fertilizer optimization, organic/natural alternatives, soil amendments, and crop-season recommendations.

```
Soil Data + Crop + Location + Season + Agricultural Knowledge
↓
Soil analysis → Deficiency detection → Crop suitability
↓
Fertilizer / amendment recommendation
```

Agent 2 — Weather & Application Timing Agent

Agent 2 handles time-sensitive conditions. It receives location, crop, recommended application, and current/forecast weather, then evaluates precipitation, temperature, humidity, wind and other relevant forecast variables to determine whether the proposed application window is appropriate.

```
Location + Crop + Recommendation
↓
Live Weather API
↓
Rain / Temperature / Humidity / Wind / Forecast
↓
Application-window assessment
```

5. Role of the LLM

The LLM should not be responsible for everything. The architecture separates deterministic calculations and external facts from language reasoning and explanation.

```
Soil data + RAG evidence + Agent outputs + Weather
↓
LLM
↓
Explanation + conversational response
```

• Understand the user's question.

• Reason over retrieved evidence and structured agent outputs.

• Explain the recommendation in farmer-friendly language.

• Answer follow-up questions.

• Clearly communicate uncertainty or missing information.

6. RAG Knowledge System

RAG (Retrieval-Augmented Generation) grounds the LLM in relevant agricultural documents instead of asking the model to rely only on its internal knowledge.

```
Authoritative PDF / Document
↓
Text extraction
↓
Cleaning & chunking
↓
Embeddings
↓
ChromaDB
```

```
↓
Semantic retrieval
↓
Relevant evidence
↓
LLM / Agent reasoning
```

The current project already contains the first RAG foundation: PDF extraction, chunking, embedding generation and ChromaDB indexing. We are now improving document layout handling and retrieval quality so that question/answer content from the government PDF is not incorrectly split.

7. Knowledge Base

• **Soil:** Soil Health Card information, NPK, pH, EC, organic carbon, micronutrients and amendments.

• **Crops:** crop requirements, soil compatibility, pH, nutrient requirements, season and water needs.

• **Fertilizers:** nutrient composition, suitability, application considerations and authoritative recommendations.

• **Organic practices:** compost, farmyard manure, green manure, crop residues and organic-matter improvement.

• **Pest management:** authoritative crop/pest information and validated management approaches.

8. Soil Report Processing

When a farmer uploads a PDF, the backend extracts the text and converts useful soil parameters into structured data. A representative structure is:

```
{
"nitrogen": {"value": 180, "unit": "kg/ha"},
"phosphorus": {"value": 15, "unit": "kg/ha"},
"potassium": {"value": 260, "unit": "kg/ha"},
"ph": {"value": 6.4},
"organic_carbon": {"value": 0.45, "unit": "%"}
}
```

The structured representation is then used by the recommendation layer instead of forcing every component to interpret raw PDF text repeatedly.

9. Deterministic Recommendation Engine

A major design principle is that the LLM should not perform all agricultural calculations. We will build a deterministic recommendation/optimization component for nutrient checks, crop scoring, constraints and validated calculations.

```
Soil data + Crop + Authoritative rules/evidence
↓
Recommendation Engine
↓
Deficiencies + Candidate options + Constraints
↓
LLM explanation
```

Exact nutrient thresholds and dosage formulas should come from validated agricultural sources or official recommendation systems. The LLM should not invent dosage values.

10. Crop Recommendation

The system will support two modes:

• **Selected crop:** determine whether the crop fits the soil and what management considerations apply.

• **Crop discovery:** rank candidate crops using soil suitability, location, season and crop requirements.

```
Soil → pH compatibility → Nutrient compatibility → Season → Location
→ Crop requirements → Suitability ranking
```

11. Weather-Aware Application Timing

A key feature is to distinguish between a recommendation and the time at which it should be applied. Agent 1 can determine that an intervention may be needed; Agent 2 can check the live forecast before suggesting an application window.

```
Recommendation: YES
Application today: WAIT
Reason: unsuitable forecast conditions
Next window: determined from forecast + validated rules
```

12. Weather API Integration

The planned development weather provider is Open-Meteo. The system can request forecast variables such as temperature, precipitation, precipitation probability, humidity and wind. The weather service will be isolated behind a Python service so that another provider can be substituted later.

```
FastAPI Backend
↓
Weather Service
↓
Open-Meteo API
↓
Forecast data
↓
Weather Agent
```

13. Pesticide / Pest Management

A future module will connect crop/pest information with authoritative recommendations and weather-aware timing. The system should not freely invent pesticide names, dosages or mixing instructions. Chemical recommendations should be grounded in current authoritative sources and presented with appropriate caution.

14. Why This Is More Than a Normal RAG Chatbot

```
Basic RAG:
PDF → Chunk → Retrieve → Answer

AgriMind:
Soil PDF → Structured Soil Data
↓
Agent Orchestrator
■ ■
Soil/Crop Agent Weather Agent
↓ ↓
RAG Live Weather API
■ ■
```

```
Recommendation Engine
↓
LLM
↓
What? / When? / Why?
↓
Farmer Dashboard
```

15. Explainable Recommendations

Every important recommendation should ideally expose:

• Recommendation

• Reason based on soil/crop information

• Retrieved evidence/source

• Weather factor, when relevant

• Confidence or evidence strength

• Source/page/section information where available

This creates a 'Why this recommendation?' feature and makes the system easier to audit, demonstrate and defend.

16. Confidence and Contradiction Handling

The system should detect incomplete or contradictory inputs instead of blindly answering.

```
Example:
Soil report → high potassium
User → 'Should I add potassium fertilizer?'

System → flag potential contradiction
→ retrieve crop-specific evidence
→ avoid unsupported additional recommendation
```

Similarly, if important soil values are missing, the system can report lower confidence and ask for the missing information.

17. Farmer-Friendly Interface

```
AGRIMIND
AI Soil & Farm Recommendation

Upload Soil Report [Choose PDF]
Location [Nagpur, Maharashtra]
Crop [Tomato ▼]
[ ANALYZE ]

SOIL HEALTH
Nitrogen LOW
Phosphorus MEDIUM
Potassium HIGH
pH 6.4
Organic Carbon LOW
```

```
RECOMMENDATION
...

WEATHER
Today / Tomorrow / Next suitable window

WHY THIS RECOMMENDATION?
...

Ask AgriMind: [ chat ]
```

18. Technology Stack

Layer	Technology
Programming	Python 3.12
Backend	FastAPI
API documentation	Swagger / OpenAPI
PDF extraction	PyMuPDF
Text processing	Python / Regex
Embeddings	Sentence Transformers
Vector database	ChromaDB
RAG	Custom Python RAG pipeline
LLM	OpenAI API or local LLM
Weather	Open-Meteo API
Database	PostgreSQL (planned)
Cache	Redis (later)
Frontend	React
UI	Tailwind CSS
Maps	Leaflet + geocoding
Authentication	JWT/OAuth (later)
Deployment	Docker + cloud hosting
Testing	Pytest + API testing
Version control	Git + GitHub
Configuration	.env environment variables

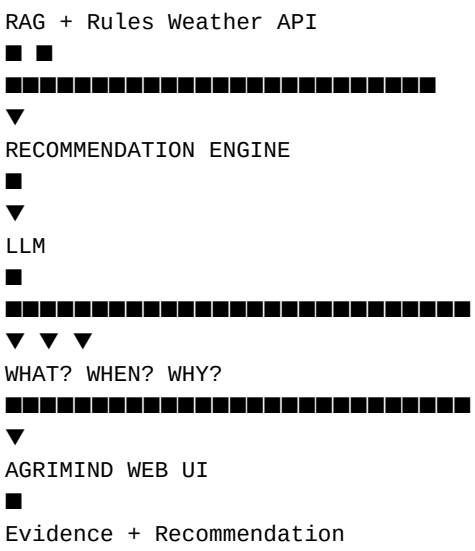
19. Backend Architecture

```
backend/
■■■ app/
■ ■■■ main.py
■ ■■■ api/
■ ■ ■■■ soil.py
■ ■ ■■■ recommendation.py
■ ■ ■■■ weather.py
■ ■ ■■■ chat.py
■ ■■■ agents/
■ ■ ■■■ soil_agent.py
```

```
■ ■ ■■■ weather_agent.py
■ ■ ■■■ orchestrator.py
■ ■■■ rag/
■ ■ ■■■ knowledge_loader.py
■ ■ ■■■ chunker.py
■ ■ ■■■ embedding_service.py
■ ■ ■■■ vector_store.py
■ ■ ■■■ retriever.py
■ ■ ■■■ index_knowledge.py
■ ■■■ services/
■ ■ ■■■ pdf_service.py
■ ■ ■■■ soil_parser.py
■ ■ ■■■ weather_service.py
■ ■ ■■■ fertilizer_service.py
■ ■ ■■■ crop_service.py
■ ■ ■■■ llm_service.py
■ ■■■ engine/
■ ■ ■■■ fertilizer_optimizer.py
■ ■ ■■■ crop_scorer.py
■ ■ ■■■ application_window.py
■ ■■■ models/
■ ■■■ soil.py
■ ■■■ crop.py
■ ■■■ recommendation.py
■■■ data/knowledge/
■ ■■■ soil/
■ ■■■ crops/
■ ■■■ fertilizer/
■ ■■■ organic/
■ ■■■ pesticides/
■■■ vector_db/
■■■ uploads/
■■■ tests/
■■■ requirements.txt
■■■ .env
■■■ Dockerfile
```

20. Complete System Architecture

```
AGRIMIND
■
FARMER / USER
■
■■■
```



21. Development Roadmap

Phase	Component	Work
Phase 1	Backend foundation	Python 3.12, virtual environment, FastAPI, Swagger — completed.
Phase 2	Soil processing	PDF upload, extraction, soil parser and structured data — basic version completed.
Phase 3	RAG foundation	Extraction, chunking, embeddings and ChromaDB — built; quality improvements in progress.
Phase 4	High-quality RAG	Fix layout extraction, preserve FAQ question/answer pairs, improve metadata and evaluate retrieva
Phase 5	Agent 1	Soil analysis, nutrient deficiency, crop suitability, fertilizer and organic recommendations.
Phase 6	Agent 2	Location, weather API, forecast analysis and application-window assessment.
Phase 7	LLM integration	Combine retrieved evidence, agent outputs and user questions into grounded explanations.
Phase 8	Chatbot/UI	React dashboard, chat interface, recommendation cards, evidence and weather window.
Phase 9	Advanced features	Crop calendar, seasonal planning, pest management, confidence, contradiction detection and multi
Phase 10	Deployment	Docker, cloud services, database, security, monitoring and production hardening.

22. Important Engineering Principles

• **No Computer Vision in the core system.** Soil reports are handled through PDF/text extraction.

• **Do not let the LLM invent agricultural facts.** Use authoritative retrieval and deterministic rules where possible.

• **Separate facts from explanations.** External APIs and validated knowledge provide facts; the LLM explains them.

• **Keep services modular.** Weather provider, LLM provider and database should be replaceable.

• **Evaluate RAG before building agents.** Retrieval quality is a foundation for reliable downstream recommendations.

• **Show evidence.** Important recommendations should expose their supporting source and reasoning.

• **Handle uncertainty.** Missing or contradictory data should lower confidence or trigger a clarification request.

23. Project Differentiation

The core differentiator is the combination of structured soil intelligence, evidence-grounded RAG, specialized agents, deterministic recommendation logic, and live weather-aware timing.

```
Not just:
PDF → Chatbot

But:
Soil → Understand → Retrieve Evidence → Reason → Validate →
Weather Check → Recommend → Explain → Show Sources
```

24. Final Project Definition

AgriMind is an evidence-grounded, multi-agent agricultural decision-support system that converts soil data, crop requirements, authoritative agricultural knowledge and live weather information into explainable, weather-aware farming recommendations.

The system is designed to answer not only **what** a farmer should consider, but also **why** the recommendation was made and **when** the action may be appropriate, while explicitly handling missing, conflicting or insufficient information.

25. Current Development Status

At the current stage, the FastAPI backend, soil PDF processing, soil parser, knowledge-document extraction, chunking, embedding service and ChromaDB indexing/retrieval tests have been established. The immediate technical priority is to improve the PDF extraction/chunking so that two-column FAQ content remains semantically coherent before we build the agent layers.

26. Next Step

The next implementation step is therefore not to add random features. We will first make the RAG pipeline reliable: clean extraction → correct question/answer units → meaningful chunks → metadata → retrieval evaluation. Once that foundation passes our test questions, we will proceed to Agent 1, then Agent 2, then the LLM orchestration and frontend.

> **Note:** This document describes the planned project architecture and implementation direction. Agricultural recommendations, fertilizer quantities, pesticide information and other high-impact farming guidance should be grounded in authoritative and current agricultural sources rather than generated solely from general LLM knowledge.